

A Rudimentary Intro to C programming

Wayne Goddard

School of Computing, Clemson University, 2025

Part 4: Pointers and Structs

17	Pointers	D1
18	Strings and Pointers	D3
19	More on Pointers	D6
20	Structs	D8
21	A Taste of Linked Lists	D11

Pointers

17.1 Computer Memory

We have thus far ignored the details of how variables are stored in memory. Indeed, in some high-level languages this ignorance is encouraged. But in C one is encouraged to know and exploit how variables are stored.

Memory is arranged as a collection of cells numbered from 0 up to how ever many is available on the machine. A single variable is stored in a cell, or in a block of cells, depending on its size. We shall see that a cell is big enough to store a single character; but a string is stored in a block of cells. Indeed, any array is stored as a block of cells.

So what happens when we define a variable: say `int A`? The compiler and operating system together choose a memory cell to store `A`—you can think of a name tag stuck on the cell door. But this cell also has a number—what we call the *address* of `A`. We have already seen that `&A` will give us this address. But other than passing that information to `scanf`, we have not used this address.

17.2 Pointers, Addresses and Dereferences

The language C has a data-type called a *pointer*. This is a variable that stores the address of another variable. Pointers use the `*` notation. In fact pointer syntax uses stars in two *different* meanings. We declare that a variable is a pointer by prefacing it with a star:

```
int *p;
```

Note that while an address is really just a number, a pointer holds the address of a variable of a specific type. That is, the pointer `p` stores the address of a variable of type `int`. (Well, actually, at the start `p` contains garbage, until we initialize it.) We say it points to a specific cell in memory.

We initialize a pointer by setting it to the address of a variable. This can be done in *two* ways: either set it to the address of some variable using the `&` notation, or set it to another pointer. Here is the former:

```
int *p = &A;
```

Now, we need to be able to work with the variable that `p` points to. This value is accessed by `*p` (sometimes called *dereferencing*).

The following code creates a variable `x` and a pointer `p` to it. Once that is set up, changing the value of `*p` changes the value of `x` and vice versa.

```

int x = 2;
int *p = &x;
printf("%d",*p); // prints 2
*p = 11;
printf("%d",x); // prints 11
x = -5;
printf("%d",*p); // prints -5 because p still points to x

```

17.3 Pointer Arithmetic and Arrays

Incrementing a pointer moves it to the next “cell” in memory. (The actual change in the address depends on the size of the variable that the pointer points to.) This pointer arithmetic is useful in array manipulation. The pointer can be initialized with the name of the array—because the name of the array is equivalent to the address of the first cell. Indeed, the name of the array is a pointer.

The following code prints out the array:

```

int B[] = {1,4,9,16};
int *p = B; // initialize p to point to start of array B
for(int i=0; i<4; i++) {
    printf("%d", *p);
    p++;
}

```

It is good practice to set unused pointers to NULL (a special constant defined in C to be equal to 0), to emphasize that we know it’s not yet or no longer usable.

CHAPTER 18

Strings and Pointers

18.1 Strings via Pointers

We have seen that in C a string is stored as an array of `chars`. It turns out that any block of `chars` can be interpreted as a string. This block can be accessed via the array name, or more generally, via a pointer.

In general, a string function can take as its argument either an array name or a pointer to the first char in the string. Here is another implementation of the function to calculate the length of a string, using pointer arithmetic.

```
int strlen(char *s) {
    int len=0;
    while (*s != '\0') {
        s++; len++;
    }
    return len;
}
```

We can use pointer arithmetic to achieve the same effect:

```
int strlen(char *s) {
    char *t = s;
    while (*t != '\0')
        t++;
    return (t-s);
}
```

Here is a further implementation of `strlen` that uses the fact that the `'\0'` character is equal to zero when used in a calculation. That is, the boolean condition `*s` is true if `*s` is nonzero, meaning `s` points to anything except the `'\0'` character. It also uses the comma construct, which for example allows two statements to be included in the update portion of a `for` loop.

```
int strlen(char *s) {
    int x=0;
    for ( ; *s; s++, x++ ) // notice the comma
        ;                // empty body
    return x;
}
```

So, most of the time when you see `char *P` written, the pointer `P` is going to point to (the first char in) some string. But sometimes it really points only to a single char. Only later code (or comments!) can clarify which.

18.2 Example Program: `strstr.c`

The following program contains a function that searches for the first occurrence of one string (called `needle`) inside another (called `haystack`).

```
// strstr.c - adapted from nkraft
// Find first occurrence of substring needle in string haystack.
#include <stdio.h>

// The function returns pointer to beginning of substring,
// or NULL if substring is not found.
char *strstr ( char *haystack, char *needle ) {
    char *start;
    for(start = haystack; *start != '\0'; start++ ) {
        char *p = needle;
        char *q = start;
        while ( *p != '\0' && *q != '\0' && *p == *q ) {
            p++;
            q++;
        }
        if( *p == '\0' )
            return start;    // reached end of needle without mismatch
    }
    return NULL;
}

int main ( void ) {
    char *one = "concatenate";
    char *two = "cat";

    char *ans = strstr( one, two );
    if ( ans == NULL )
        printf( "needle not found\n" );
    else
        printf( "needle starts in position %d of haystack\n", ans-one );
}
```

```
    return 0;  
}
```

18.3 Changing Strings

One can also write code that alters a string. For example, what does the following code do to a string that was defined by `char *beth`?

```
while( *beth==' ' )  
    beth++;
```

It removes any leading spaces.

But note that this assumes that `beth` is stored in an array. If one writes a declaration as `char *beth = " happy"`, the string “happy” is a constant and not alterable (you’ll get a segmentation fault).

CHAPTER 19

More on Pointers

19.1 Pass by Value

In C, all function arguments are passed by value. That is, the function receives a copy of the value, and cannot change the original. So while the contents of an array that is passed to a function can change, ints passed to a function cannot change. And yet `scanf` does its job...

The solution is that you can pass the function the *address* of a variable. For example, a function to swap the values of two numbers uses pointers:

```
// swaps the values pointed to by a and b
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

The calling code might look something like:

```
int x,y;
// x and y initialized
swap( &x, &y );
```

An array is automatically passed as an address, equivalent to a *pointer* to the base type.

19.2 Dynamic Memory Allocation

One can create an array on the fly: this uses dynamic memory allocation. This is especially important for arrays (and more advanced structures) that exist only temporarily.

The function `calloc` is contained in `stdlib.h`. It reserves/allocates and initializes a block of memory. The function takes two `int` arguments: the first argument is the number of elements to be stored and the second argument is the size in bytes (think “cells”) of the element that is to be stored (which is obtained using the `sizeof`). The `calloc` function returns the address of the beginning of the block.

```
int *intPtr;  
intPtr = calloc( 1000, sizeof(int));  
*intPtr = 10;
```

will create an array with 1000 elements and set the first value to 10 (the rest being zero).

Memory that is allocated with `calloc` must be released afterwards using the `free` command: `free(intPtr);`. This allows memory to be recycled.

Structs

Structs are used for grouping. Sometimes called a *record*, they are the precursor to objects, the cornerstone of C++.

20.1 Defining and Accessing a Struct

A struct is a collection of *members*. There are a couple of ways to set up a struct. One way is to use `typedef`, which is used in general to create a user-defined type. The following example creates a `Date` type which can be used in a program.

```
typedef struct {
    int day;
    char month[10];
    int year;
} Date;
```

One has in general direct member access. This uses DOT notation:

```
Date D;
D.day = 1;
D.month = "January";
D.year = 2008;
```

You can have an array of structs. (Indeed you can have an array of anything.) So a program might care about a sequence of events, stored as `Date dates[100]`

20.2 Structs and Pointers

A struct can be passed as a parameter to a function. If so, it should be passed by reference; that is, the function receives a pointer to the struct.

```
void increment(Date *dt) {
    (*dt).day ++;
    // some tests for reached end of month etc...
}

...
Date D;
...
increment(&D);
```

The brackets are necessary; thus there is a common shorthand: `dt->day`

Note that one often ends up with structs dynamically allocated. The function `malloc` is similar to `calloc`, except that it takes only one argument, the number of bytes needed.

```
Date *Q;
Q = (Date *) malloc(sizeof(Date));
```

20.3 Example Code: student.c

```
#include<stdio.h>
#include<stdlib.h>

#define MAX_STUD 20
#define MAX_LEN 20

typedef struct
{
    char firstName[MAX_LEN];
    char lastName[MAX_LEN];
    float grade;
} Student;

typedef struct
{
    int numStuds;
    Student stud[MAX_STUD];
    float average;
} Class;

void readClass(Class*);
void writeClass(Class*);

int main() {
    Class *r;
    r = (Class *)malloc(sizeof(Class));
    readClass(r);
    writeClass(r);
    return 0;
}
```

```
}

void readClass(Class *C) {
    scanf("%d",&C->numStuds);
    int s;
    float total=0.0;
    for(s=0; s<C->numStuds; s++) {
        scanf("%s%s%f", &C->stud[s].firstName, &C->stud[s].lastName,
                                                    &C->stud[s].grade);

        total += C->stud[s].grade;
    }
    C->average = total/C->numStuds;
}

void writeClass(Class *C) {
    int s;
    for(s=0; s<C->numStuds; s++) {
        printf("%s %s got %.2f\n", C->stud[s].firstName, C->stud[s].lastName,
                                                    C->stud[s].grade);
    }
    printf("Class average is %.2f\n",C->average);
}
```

A Taste of Linked Lists

We consider here one of the standard uses for dynamically created structs: namely, what is called a linked list. A linked list is an example of a data structure: it manages data in a way that is suitable for some tasks. An array is the commonest data structure. In some cases linked lists are a replacement for arrays, especially when the actual number of items is unknown at the beginning and can vary considerably.

The point of the chapter is to give an example, leaving most of the details of linked lists for a later time.

21.1 Linked List

A *linked list* consists of nodes with one node pointing to the next. The simplest (and most common) version is implemented with:

- one pointer to the first node (called the head)
- each node being a `struct` with data and a pointer called `next`
- the last node's next is `NULL`

The procedures for insert and deletion in general is more complicated than for arrays. For our sample code we consider only one form of insertion. On the other hand, iterating through a linked list is almost the same as it is in arrays. Here is the pseudocode:

```
void print( )
{
    node * curr = head;
    while( curr != NULL ) {
        print curr -> data;
        curr = curr -> next;
    }
}
```

21.2 Example Code: `linkedlist.c`

The goal is to read in some words and then print them out. This obviously could be done with an array. We do it with a linked list for show.

We are going to keep the list in the order that the names are presented. Thus at each step we add to the end of the current list. To make things easier, we maintain a

pointer that points to the last node in the list. Note that the first insertion changes the value of head; thereafter it never changes.

```
#include <stdlib.h>
#include <stdio.h>

typedef struct nodeType {
    char data[100];
    struct nodeType * next;
} node;

node * head;
node * tail;

void readWords( );
void printWords( );

int main( void )
{
    readWords( );
    printWords( );
    return 0;
}

void readWords( )
{
    printf("How many words?\n" );
    int count;
    scanf("%d", &count);
    for(int n=1; n<=count; n++) {
        node * newNode = (node*) malloc( sizeof(node) );
        newNode -> next = NULL;
        printf("Enter word %d: ", n );
        scanf("%s", newNode->data );
        if( n==1 ) {
            head = tail = newNode;
        }
        else {
            tail->next = newNode;
        }
    }
}
```

```
        tail = tail->next;
    }
}

void printWords ( )
{
    node * curr = head;
    while( curr != NULL ) {
        printf("%s\n", curr -> data );
        curr = curr -> next;
    }
}
```